A D PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)

PASSWORD LAB

A Programming with Python Mini Project Report

A project report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Information Technology

Semester V

Submitted by

Ayush Asodariya

Enrollment No.
12502080603002

Faculty Guide

Prof. Nayan Mali

Academic Year 2026–2027 (Odd)

Department of Information Technology
New Vallabh Vidyanagar, Anand, Gujarat, India

A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

CERTIFICATE

This is to certify that the project work entitled “Password Lab” has been carried out by Ayush Asodariya, Enrollment No. 12502080603002, a student of B.Tech Information Technology at A. D. Patel Institute of Technology (ADIT), CVM University, in partial fulfilment of the requirements of the course Programming with Python (202044504) during the academic year 2026–27.

The work is original, has been completed under my guidance, and has not been submitted elsewhere for any other degree or diploma.

Project Guide
Nayan Mali
Dept. of Information Technology, ADIT

Head of Department
Priyang Bhatt
Dept. of Information Technology, ADIT

\

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for the valuable guidance, encouragement and constructive feedback given throughout this project. I am thankful to the Head of the Department of Information Technology and to the faculty of A. D. Patel Institute of Technology for providing the facilities and a supportive learning environment.

I also thank my classmates and friends for testing the application and sharing useful suggestions, and my family for their constant support and motivation.

Ayush Asodariya
Enrollment No. 12502080603002

1. Introduction

1.1 Background and Problem Statement

Passwords are still the main way people protect their accounts. Attackers do not guess randomly; they try lists of common passwords, predictable patterns such as “Password123!”, keyboard walks such as “1qaz2wsx”, and personal details such as names and birth years. A password that merely satisfies “one capital, one digit, one symbol” rules can still be very easy to guess. Users therefore need a tool that explains why a password is weak and helps them create a better one, without sending the password to anyone.

1.2 Objectives

- Build a checker that rates any password from 0 to 100 and explains every weakness it finds.
- Build a generator that produces strong random passwords and memorable passphrases using a secure random source.
- Keep everything local: no network requests, no logs, no saved passwords.
- Keep the code modular and verify it with automated tests.

1.3 Scope

A single-user desktop tool for Windows, macOS and Linux (Python 3.10+). It does not store passwords, manage accounts or query online breach databases, and its score is a heuristic, not a guarantee of safety.

2. Tools and Technologies

Technology	Purpose in this project
Python 3.10+	Main language; standard library only, so no installation of extra packages is needed.
Tkinter / ttk	Graphical interface: tabs, entry fields, progress bar, checklist and dialogs.
re (regular expressions)	Detects character groups, repeated runs, repeated pairs, years and word-plus-suffix patterns.
secrets	Cryptographically secure random choice and shuffling for generation.
threading and queue	Runs analysis off the interface thread and passes results back safely.
dataclasses	Finding and Analysis objects that carry structured results.
unittest	Automated testing of the analyzer and generator.

3. System Design

3.1 Architecture

The GUI contains no scoring or generation logic; it only collects input, calls the two logic modules and displays their results, so the logic can be tested without opening a window.

Module	Main functions	Responsibility
main.py	PasswordLabApp	Builds both tabs, debounces typing, runs the worker thread, renders score, checklist and suggestions.
analyzer.py	analyze_password()	Applies all rules, computes score, label and entropy estimate, returns an Analysis object.
generator.py	generate_password(), generate_passphrase()	Creates random passwords and passphrases with input validation.
test_password_lab.py	15 test methods	Verifies detection rules, edge cases and generator behaviour.

Table 1: Module overview

3.2 Data Flow

When the user types, the GUI waits 180 ms (debounce), then starts a worker thread that calls analyze_password(). The result is placed in a queue; the interface polls the queue every 100 ms and displays the result only if the text box still contains the same password, so stale results are never shown.

3.3 Scoring Model

The score is built in four transparent steps:

- **Add:** Points are added for length (2 per character, maximum 35), character variety (7 per extra group, maximum 20), and length bonuses (+8 at 20 characters, +5 more at 28).
- **Subtract:** Each finding subtracts points according to its category and severity. For example, a blocklist match costs 55, personal information 35, a predictable pattern 20, and a keyboard walk 18. A warning-level finding costs half of the category value.
- **Cap:** A blocklisted password cannot score above 15, and a password under 8 characters cannot score above 20.
- **Label:** 0–19 Very Weak, 20–39 Weak, 40–59 Fair, 60–79 Strong, 80–100 Very Strong.

The entropy estimate is length × $\log_2$(pool size), where the pool is the total size of the character groups present, reduced when predictable patterns are found. It is displayed as an estimate only.

4. Implementation

4.1 Strength Analyzer (analyzer.py)

The analyzer is one function, analyze_password(password, personal_info, blocklist), which never modifies the password. It runs these checks in order:

1. Length and character groups using regular expressions such as [A-Z], [a-z], \d and [^A-Za-z0-9\s].
2. Blocklist match, including leetspeak: characters such as @, 0, 3, $ and 1 are translated back to letters before comparing, so “P@ssw0rd” is caught.
3. Repetition: runs like “aaaa” via (.)\1{2,}, repeated pairs like “abab”, and repeated substrings like “abcabcabc”.
4. Sequences such as “abcd” or “4321”, and keyboard patterns by sliding a window over the keyboard rows (also reversed) plus known walks like “1qaz2wsx”.
5. Common words and names, year-like numbers, and word-plus-digits patterns.
6. Personal information supplied by the user, also compared after leetspeak translation.

Every problem becomes a Finding with a category, message, severity (info, warning or critical) and a suggestion, so the user always sees what is wrong and how to fix it.

4.2 Generator (generator.py)

generate_password() builds one pool for each enabled group, optionally removes ambiguous characters (O, 0, l, 1 and similar), picks one character from every group so that each chosen group is guaranteed to appear, fills the remaining length from the combined pool with secrets.choice, and finally shuffles with secrets.SystemRandom(). Length must be 8–128; invalid settings raise ValueError. generate_passphrase() picks 3–10 words from a 168-word list with secrets.choice, joins them with a hyphen and optionally appends a two-digit number. A 20-character password using all four groups has roughly 128 bits of entropy.

4.3 User Interface (main.py)

The interface has two tabs. The Strength checker tab has a masked password field (Show/Hide, Clear), an optional personal-details field, a colour-coded score with progress bar and entropy figure, a checklist, and a list of detected issues with suggestions. The Password generator tab has length, group and count controls, scored suggestions with Copy buttons, and a passphrase section.

5. Testing and Results

5.1 Unit Testing

The project includes 15 unit tests written with unittest. All 15 pass (run with python -m unittest -v). They cover an empty password, a very short password, a 120-character password that must not be truncated, a blocklisted password (score must be 15 or lower), repetition, sequence, keyboard walk, dictionary word, leetspeak, Unicode, whitespace, personal information, generated password length and groups, passphrase word count, and invalid generator settings.

5.2 Sample Analysis Results (real analyzer output)

Input	Score	Label	Entropy (bits)	Findings
password	0	Very Weak	7.5	Blocklist, dictionary, predictable
abc	0	Very Weak	14.1	Too short, weak composition
1qaz2wsx!	16	Very Weak	44.0	Keyboard walk
P@ssw0rd123!	0	Very Weak	15.8	Leetspeak form of blocklisted word
WelcomeSunshine123!	16	Very Weak	49.8	Common words, predictable
Tr9#kLm2$vQx8!pZ	52	Fair	105.1	None
correct-horse-battery-staple-92	62	Strong	189.4	None

Table 2: Sample analyzer output

The results show that the checker is not fooled by simple tricks: adding a capital letter, digits and a symbol to a common word (“P@ssw0rd123!”, “WelcomeSunshine123!”) still leaves it Very Weak, while a random 16-character password or a long passphrase scores well.

6. Limitations and Future Scope

6.1 Limitations

- The score is a heuristic and not a measurement of real-world guess resistance.
- The blocklist has only 31 entries and the word lists are small offline samples, not a breach database.
- The entropy estimate treats a passphrase like random characters, so it overstates word-based passwords. A 5-word passphrase from the built-in 168-word list gives only about 43 bits of real randomness.
- A random password shorter than 20 characters can score at most 55 (Fair), because the length bonus starts at 20.

6.2 Future Scope

- Use a larger audited word list (such as 7,776 words, about 12.9 bits per word) and a load-from-file blocklist.
- Add an optional breached-password check using a k-anonymity range query, which sends only a short hash prefix and never the password.
- Add a more realistic entropy model, automatic clipboard clearing and a packaged Windows executable.